

PROGRAMMATION GRAPHIQUE — Vulkan

Pipeline GPU Explicite, SPIR-V & Rendu Moderne — C++ / Jenga / NKWindow / NKMath

UNITÉ	NIVEAU	DURÉE	BUILD
Vulkan 1.2 → 1.3	Intermédiaire → Avancé	~140 h — 18 semaines	Jenga 2.0.1 + VulkanSDK + NKWindow

PHILOSOPHIE

Vulkan est une API graphique bas-niveau qui expose directement le GPU. Contrairement à OpenGL, rien n'est implicite : l'étudiant gère explicitement la mémoire GPU, la synchronisation, les render passes et les command buffers. Ce cours part de zéro en Vulkan et construit progressivement un moteur de rendu 3D moderne. Jenga gère le build multi-plateforme, NKWindow crée la surface Vulkan (VkSurfaceKHR), NKMath (construite par les étudiants) fournit les maths. Objectif : comprendre POURQUOI chaque concept Vulkan existe et comment les drivers l'implémentent.

POSITIONNEMENT & DIFFÉRENCES AVEC OPENGL

Aspect	OpenGL	Vulkan
Initialisation	~10 lignes + contexte implicite	~500 lignes, tout explicite
Gestion mémoire	Driver alloue automatiquement	Manuelle : VkDeviceMemory, pools
Synchronisation	Implicite (glFinish/glFlush)	Explicite : semaphores, fences, barriers
Shaders	GLSL compilé à la volée	SPIR-V (binaire), pré-compilé offline
Multi-threading	Non-thread-safe (contexte par thread)	Conçu pour multi-thread
Validation	glGetError() synchrone	Validation layers optionnelles
Render Pass	Implicite	VkRenderPass décrit explicitement les passes
Pipeline state	Modifié par des appels d'état	VkPipeline : objet immutable pré-compilé
CPU overhead	Élevé (driver fait beaucoup)	Minimal (l'app fait tout)

ÉCOSYSTÈME TECHNIQUE

1. Jenga — Build System avec VulkanSDK

```
# TP Vulkan type - fichier .jenga
from Jenga import *

with workspace('TP_VK_XX', location='.'):
    configurations(['Debug', 'Release'])
    targetoses([TargetOS.WINDOWS, TargetOS.LINUX, TargetOS.MACOS])
    targetarchs([TargetArch.X86_64])

    with project('TP_VK'):
        windowedapp()
        language('C++')
        cppdialect('C++17')
        files(['src/**/*.cpp', 'nkmath/**/*.cpp'])
        includedirs(['include', 'nkmath',
                     '%{env.VULKAN_SDK}/include',
                     '%{env.NKENTSEU_ROOT}/include'])
        links(['NKWindow', 'NKCore', 'NKContainers'])
        libdirs(['%{env.NKENTSEU_ROOT}/lib',
                 '%{env.VULKAN_SDK}/lib'])
        pchheader('pch.h')
        pchsource('src/pch.cpp')

        with filter('system:Windows'):
            links(['vulkan-1', 'user32', 'gdi32'])
            defines(['VK_USE_PLATFORM_WIN32_KHR',
                    'NKENTSEU_PLATFORM_WINDOWS'])
        with filter('system:Linux'):
            links(['vulkan', 'X11', 'pthread', 'dl'])
            defines(['VK_USE_PLATFORM_XLIB_KHR',
                    'NKENTSEU_PLATFORM_LINUX'])
        with filter('system:macOS'):
            links(['vulkan'])
            defines(['VK_USE_PLATFORM_METAL_EXT',
                    'NKENTSEU_PLATFORM_MACOS'])
        with filter('configurations:Debug'):
            defines(['DEBUG', 'VK_VALIDATION']); symbols('On')
        with filter('configurations:Release'):
            defines(['NDEBUG']); optimize('Speed')

        # Shaders SPIR-V : compiler en pre-build step
        # glslc shaders/xxx.vert -o shaders/xxx.vert.spv
        prebuildcommands([
            'glslc shaders/%.vert -o shaders/%.vert.spv',
            'glslc shaders/%.frag -o shaders/%.frag.spv',
        ])
    )
```

2. NKWindow — Surface Vulkan

NKWindow crée la fenêtre native et expose une `VkSurfaceKHR` via `NkWindow::CreateVulkanSurface()`. L'étudiant gère l'instance, le device et la swapchain.

```

// Intégration NKWindow + Vulkan
NkWindowConfig cfg;
cfg.title      = NkString("TP Vulkan");
cfg.width      = 1280; cfg.height = 720;
cfg.useVulkan  = true;    // signale à NKWindow de préparer une surface VK

NkWindow window; window.Create(cfg);

// Récupérer les extensions requises par NKWindow
uint32_t extCount = 0;
const char** extensions = window.GetVulkanInstanceExtensions(&extCount);

// L'étudiant crée VkInstance avec ces extensions (voir M01)

// Créer la surface VkSurfaceKHR via NKWindow
VkSurfaceKHR surface;
window.CreateVulkanSurface(instance, nullptr, &surface);

// Boucle principale
NkEventSystem events; events.Init(); events.RegisterWindow(&window);
while (window.IsOpen()) {
    events.PollEvents();
    // acquérir swapchain image, submit, present ...
}

```

3. NKMath — Construite par les Étudiants

Type NKMath	Contenu	Usage Vulkan	Module
NkVec2f / NkVec2i	Vecteur 2D float/int, dot, normalize	UV, viewport offset	M-01
NkVec3f	Vecteur 3D, cross, normalize, reflect	Position, normale	M-02
NkVec4f	Vecteur 4D homogène	Push constants, RGBA	M-03
NkMat4x4	Matrice 4x4 column-major, MVP, lookAt, persp	UBO, push constants	M-03
NkMathUtils	Lerp, clamp, deg2rad, pi, easing	Animations	M-01
NkQuaternion	Quaternion, slerp, toMat4	Rotations 3D	M-07
NkTransform3D	Position+Rotation+Scale → NkMat4x4	Scène graph	M-08
NkColor / NkColorF	RGBA 8-bit et float	Couleurs, clear values	M-01

PRÉREQUIS

C++

- C++ intermédiaire : classes, templates, RAII, smart pointers
- Gestion mémoire avancée : allocateurs, alignement, padding
- Multithreading C++11 : `std::thread`, `std::mutex`, `std::atomic` (concepts)
- Bitwise operations : flags, masques de bits

Mathématiques

- Algèbre linéaire complète : vecteurs 3D/4D, matrices 4×4, transformations
- Coordonnées homogènes, produit scalaire et vectoriel
- Trigonométrie, interpolation

Outils

- VulkanSDK installé (variable `$VULKAN_SDK` configurée)
- `glslc` : compilateur GLSL → SPIR-V (inclus dans VulkanSDK)
- Jenga : `pip install Jenga`
- GPU supportant Vulkan 1.1+ (tout GPU post-2016)
- NKWindow configuré (fourni par le professeur)

BIBLIOTHÈQUES AUTORISÉES

VulkanSDK (`vulkan.h`, validation layers) | `glslc` (GLSL→SPIR-V, pré-build Jenga) | `stb_image.h` (chargement textures) | NKWindow (surface Vulkan) | NKMath (étudiants) | INTERDIT : Vulkan-Hpp wrapper, VMA, vk-bootstrap, GLM, SDL, GLFW pour le rendu

VUE D'ENSEMBLE DU PROGRAMME

Sem.	Mod.	Titre	Thème	Durée
1	01	Fondations : GPU Moderne & Architecture Vulkan	Instance, Devices	8h
2	02	Swapchain, Surface & Boucle de Rendu	Présentation GPU	8h
3	03	Command Buffers & Render Passes	Enregistrement GPU	10h
4	04	Pipeline Graphique & Shaders SPIR-V	VkPipeline, GLSL→SPIR-V	10h
5	05	Vertex Buffers & Mémoire GPU	VkBuffer, VkDeviceMemory	10h
6	06	Descriptors & Uniform Buffers	UBO, VkDescriptorSet	10h
7	07	Textures & Image Samplers	VkImage, VkSampler	10h
8	08	Transformations 3D & NKMath Complète	MVP, Push Constants	8h
9	09	Depth Buffering & Multi-Pass Rendering	VkDepthImage, RenderPass	10h
10	10	Éclairage PBR en Vulkan	Phong → PBR GLSL	10h
11	11	Synchronisation Avancée & Multi-Threading	Barriers, semaphores	12h
12	12	Compute Pipelines & GPGPU Vulkan	VkComputePipeline	12h
13	13	Raytracing (VK_KHR_ray_tracing_pipeline)	TLAS/BLAS, rchit/rngen	10h
14-18	14	Projet Final	Moteur Vulkan complet	22h

Objectifs — NKMath : NkVec2f, NkColor, NkMathUtils

- Comprendre pourquoi Vulkan est explicite là où OpenGL était implicite
- Créer VkInstance avec validation layers
- Sélectionner le physical device et créer le logical device
- Configurer les queues (graphics + present + compute)

S01
THÉORIE**Architecture GPU & Pourquoi Vulkan**

- GPU moderne : command queues, shader cores, caches L1/L2, VRAM
- Vulkan vs OpenGL : overhead driver, multi-threading, prévisibilité
- VkInstance : point d'entrée — charge le driver Vulkan
- VkPhysicalDevice : énumérer les GPUs disponibles, propriétés, features
- VkDevice (logical device) : interface de commande vers le GPU choisi
- Queue Families : graphics, compute, transfer, present — choisir les bonnes
- Validation Layers : VK_LAYER_KHRONOS_validation — activer en Debug

S02
TP**TP — Instance, Device & Validation Layers**

- VkApplicationInfo → VkInstanceCreateInfo → vkCreateInstance
- Extensions d'instance : VK_EXT_debug_utils, extensions NKWindow
- VkDebugUtilsMessengerEXT : callback de validation en Debug
- vkEnumeratePhysicalDevices → sélectionner par score (GPU dédié > intégré)
- findQueueFamilies() : indices graphics + present
- vkCreateDevice avec les features requises
- vkGetDeviceQueue : récupérer les handles de queues
- NKMath : NkVec2f, NkColor, NkMathUtils

Livable : Instance Vulkan fonctionnelle + device sélectionné + validation layers actives

LIVRABLE
M01

vkCreateInstance + vkCreateDevice + validation layers Debug + vkDestroyDevice/vkDestroyInstance propres (RAII).

Objectifs

- Créer la VkSurfaceKHR via NKWindow
- Configurer et créer la swapchain (format, présentation, images)
- Implémenter la boucle acquire → render → present

S03
THÉORIE

Swapchain — Théorie

- VkSurfaceKHR : représentation de la fenêtre native pour Vulkan
- VkSwapchainKHR : file d'images présentables (double/triple buffering)
- Surface capabilities : formats supportés (VK_FORMAT_B8G8R8A8_SRGB),
- présentation modes (FIFO=vsync, MAILBOX=triple buffering)
- VkImage vs VkImageView : image GPU brute vs vue avec format et aspect
- Images de la swapchain : gérées par le driver, pas par l'app
- Semaphores vs Fences : GPU-GPU vs CPU-GPU synchronisation

S04
TP

TP — Swapchain & Premier Frame

- window.CreateVulkanSurface(instance, nullptr, &surface)
- querySwapchainSupport() : formats + present modes + capabilities
- chooseSwapFormat() : préférer VK_FORMAT_B8G8R8A8_SRGB + SRGB color space
- chooseSwapPresentMode() : MAILBOX si disponible, sinon FIFO
- chooseSwapExtent() : résolution en pixels (pas en screen coords)
- vkCreateSwapchainKHR, vkGetSwapchainImagesKHR, créer les VkImageViews
- Boucle : vkAcquireNextImageKHR → (vide pour l'instant) → vkQueuePresentKHR
- Gestion redimensionnement : VK_ERROR_OUT_OF_DATE_KHR → recréer swapchain

Livrable : Fenêtre Vulkan avec boucle de présentation fonctionnelle (fond noir)

S05
THÉORIE**Render Pass — Théorie**

- VkRenderPass : décrit les attachements (color, depth), leur format,
- leur usage en entrée/sortie (loadOp/storeOp), et les subpasses
- Subpass : groupe de draw calls partageant les mêmes attachements
- VkFramebuffer : lie les VkImageViews aux attachements du render pass
- VkAttachmentDescription : format, samples, loadOp=CLEAR/LOAD, storeOp=STORE
- Subpass dependencies : synchronisation entre subpasses (VkSubpassDependency)

S06
THÉORIE**Command Buffers — Théorie**

- VkCommandPool : pool d'allocation de command buffers (par thread, par queue)
- VkCommandBuffer : enregistre les commandes GPU
- vkBeginCommandBuffer, vkCmdBeginRenderPass, vkCmdDraw, vkCmdEndRenderPass, vkEndCommandBuffer
- Double/Triple buffering : un command buffer par swapchain image (ou par frame en vol)
- Frames en vol (MAX_FRAMES_IN_FLIGHT) : pipeliner CPU et GPU
- Secondary command buffers : enregistrement parallèle depuis plusieurs threads

S07
TP**TP — Render Pass + Clear**

- Créer VkRenderPass : 1 color attachment (format swapchain), loadOp=CLEAR
- Créer VkFramebuffers : un par swapchain image
- Créer VkCommandPool + allouer MAX_FRAMES_IN_FLIGHT command buffers
- Créer semaphores (imageAvailable, renderFinished) + fences (inFlight) par frame
- Boucle : acquire → begin CB → beginRenderPass (clearColor) → endRenderPass → end CB → submit → present
- Synchronisation propre : vkWaitForFences, vkResetFences

Livrable : Fenêtre Vulkan avec fond coloré via clear value, 60 FPS, synchronisation correcte

S08
THÉORIE**Pipeline Graphique Vulkan**

- VkGraphicsPipeline : objet immutable décrivant TOUT l'état de rendu
- Stages : vertex input, vertex shader, tessellation, geometry, rasterization, fragment shader, color blending, depth/stencil
- VkPipelineLayout : structure des descripteurs et push constants
- VkShaderModule : code SPIR-V chargé en GPU
- SPIR-V : format binaire portable pour les shaders
- glslc : glslc shader.vert -o shader.vert.spv (intégré dans Jenga prebuild)

S09
THÉORIE**Shaders GLSL pour Vulkan (SPIR-V)**

- Différences GLSL OpenGL vs Vulkan : layout(set=N, binding=M), pas de gl_FragColor
- Vertex shader : layout(location=0) in vec3 aPos → gl_Position
- Fragment shader : layout(location=0) out vec4 outColor
- Push constants : layout(push_constant) uniform PushConstants { mat4 mvp; }
- Coordonnées Vulkan : Y inversé, Z [0,1] (vs OpenGL Z [-1,1])
- Spirv-reflect : introspection des binaires SPIR-V

S10
TP**TP — Pipeline & Premier Triangle**

- glslc intégré dans Jenga prebuildcommands pour auto-compiler les shaders
- Charger SPIR-V : lire le fichier binaire .spv → vkCreateShaderModule
- Créer VkPipelineLayout (vide pour l'instant)
- Configurer tous les stages du pipeline (vertex input, rasterizer, color blend...)
- vkCreateGraphicsPipelines : compiler le PSO (Pipeline State Object)
- Enregistrer vkCmdBindPipeline + vkCmdDraw(3, 1, 0, 0) dans le command buffer
- Premier triangle (hardcodé dans le vertex shader) à l'écran

Livable : Triangle RGB dans une fenêtre Vulkan — pipeline graphique complet

LIVRABLE
M04

VkGraphicsPipeline complet + shaders SPIR-V + triangle affiché. Pipeline encapsulé dans une classe PipelineBuilder.

S11
THÉORIE**Mémoire GPU en Vulkan**

- vkGetPhysicalDeviceMemoryProperties : types de mémoire (DEVICE_LOCAL, HOST_VISIBLE, HOST_COHERENT)
- VkBuffer : objet buffer — vkCreateBuffer
- VkDeviceMemory : allocation de mémoire GPU — vkAllocateMemory
- findMemoryType() : trouver le bon type selon les flags requis
- vkBindBufferMemory : associer le buffer à la mémoire
- Staging buffer : CPU-visible → copie → DEVICE_LOCAL (rapide)
- vkMapMemory / vkUnmapMemory : accès CPU à la mémoire HOST_VISIBLE

S12
THÉORIE**Vertex Input & Index Buffers**

- VkVertexInputBindingDescription : stride, inputRate (vertex ou instance)
- VkVertexInputAttributeDescription : location, format, offset
- vkCmdBindVertexBuffers, vkCmdBindIndexBuffer
- vkCmdDrawIndexed vs vkCmdDraw
- Transfer queue : copie asynchrone staging→DEVICE_LOCAL
- VkCommandBuffer de transfert : vkCmdCopyBuffer

S13
TP**TP — Mesh avec Vertex & Index Buffers**

- Struct Vertex { NkVec3f pos; NkVec3f color; NkVec2f uv; }
- Créer staging buffer (HOST_VISIBLE|HOST_COHERENT) → copier données CPU
- Créer vertex buffer (DEVICE_LOCAL) → vkCmdCopyBuffer staging→device
- Créer index buffer de la même façon
- Configurer VkVertexInputBindingDescription et AttributeDescriptions
- Classe Buffer : encapsule VkBuffer + VkDeviceMemory + upload
- Mesh cube 3D : 8 vertices, 36 indices

Livable : Cube Vulkan avec vertex/index buffers en DEVICE_LOCAL

S14
THÉORIE**Descriptor Sets — Théorie**

- Descriptors : mécanisme Vulkan pour lier des ressources aux shaders
- VkDescriptorSetLayout : déclare les bindings (type, stage, count)
- VkDescriptorPool : pool d'allocation de descriptor sets
- VkDescriptorSet : ensemble de descriptors alloués
- vkUpdateDescriptorSets : écrire les ressources dans les descriptors
- Types : VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER, COMBINED_IMAGE_SAMPLER, STORAGE_BUFFER...
- Sets multiples : set 0 = global (caméra), set 1 = matériau, set 2 = objet

S15
THÉORIE**Uniform Buffer Objects (UBO) en Vulkan**

- UBO : buffer uniform mis à jour chaque frame (per-frame data)
- Un UBO par frame en vol (MAX_FRAMES_IN_FLIGHT) pour éviter les conflits
- Alignement GLSL std140 : les structs doivent respecter les règles d'alignement
- Push Constants : données petites (<128 bytes) transmises directement sans descriptor
- vkCmdPushConstants : plus rapide que les UBO pour les données par-objet

S16
TP**TP — UBO Camera + Push Constants**

- Struct UBOcamera { NkMat4x4 view; NkMat4x4 proj; NkVec3f camPos; }
- Créer un UBO par frame en vol (HOST_VISIBLE|HOST_COHERENT)
- VkDescriptorSetLayout : binding 0 = UBO camera (VERTEX|FRAGMENT stage)
- Créer descriptor pool et allouer descriptor sets
- vkUpdateDescriptorSets : lier les UBOs aux descriptors
- Push constants : mat4 model (par objet) via vkCmdPushConstants
- Cube rotatif avec caméra orbitale — MVP complet Vulkan

Livable : Cube 3D tournant avec MVP via UBO + push constants

LIVRABLE
M06

Descriptor system complet. NkMat4x4 complète dans NKMath (column-major). Cube 3D avec caméra orbitale.

S17
THÉORIE**Images Vulkan — Théorie**

- VkImage : image GPU — format, extent, mip levels, array layers, usage
- VkImageLayout : état de l'image (UNDEFINED, TRANSFER_DST, SHADER_READ_ONLY...)
- Image layout transitions : pipeline barriers (vkCmdPipelineBarrier)
- VkImageView : vue sur une image (aspect, mip range, array range)
- VkSampler : paramètres de sampling (filter, wrap, mip lod bias, anisotropy)
- stb_image : charger PNG/JPG → données CPU → staging → VkImage DEVICE_LOCAL
- Mipmaps : générer avec vkCmdBlitImage depuis le mip 0

S18
TP**TP — Texture 2D Vulkan**

- Charger image avec stb_image → staging buffer
- Créer VkImage (TRANSFER_DST | SAMPLED, DEVICE_LOCAL)
- Transition layout : UNDEFINED → TRANSFER_DST → SHADER_READ_ONLY
- vkCmdCopyBufferToImage : copier staging → VkImage
- Générer mipmaps : vkCmdBlitImage en boucle
- Créer VkImageView + VkSampler
- Ajouter binding 1 = COMBINED_IMAGE_SAMPLER dans le descriptor set layout
- Classe Texture2D : encapsule VkImage + VkDeviceMemory + VkImageView + VkSampler

Livable : Cube Vulkan texturé avec mipmaps et sampler anisotropique

S20
THÉORIE**Depth Buffer Vulkan**

- VkFormat pour depth : VK_FORMAT_D32_SFLOAT, VK_FORMAT_D24_UNORM_S8_UINT
- findDepthFormat() : vkGetPhysicalDeviceFormatProperties
- Créer VkImage depth (DEPTH_STENCIL_ATTACHMENT usage)
- Ajouter depth attachment au VkRenderPass
- vkClearDepthStencilValue dans le clear value array
- Pipeline depth state : VkPipelineDepthStencilStateCreateInfo

S21
THÉORIE**Render Passes Multiples**

- G-Buffer : render pass 1 → écrire position/normal/albedo dans des VkImages
- Lighting pass : render pass 2 → lire le G-Buffer, calculer l'éclairage
- Shadow pass : render pass depuis la lumière → shadow map VkImage
- VkSubpassDependency : synchroniser la fin du rendu → lecture en shader
- Attachment references : input attachments pour la communication inter-subpass

S22
TP**TP — Depth + Shadow Map**

- Ajouter depth image à la swapchain framebuffer
- Depth prepass : render pass séparé pour peupler le depth buffer
- Shadow pass : render pass avec depth-only framebuffer (shadow map)
- Main pass : lire shadow map via descriptor set pour le shadow test
- Synchronisation : VkImageMemoryBarrier entre les passes

Livrable : Scène 3D avec depth correct + shadow map basique

S25
THÉORIE**Synchronisation Vulkan**

- Semaphores : synchronisation GPU-GPU (binary + timeline semaphores)
- Fences : synchronisation GPU-CPU (CPU attend que le GPU finisse)
- Events : GPU-GPU ou CPU-GPU avec granularité fine
- Pipeline Barriers : synchroniser les accès mémoire et les layout transitions
- VkMemoryBarrier, VkBufferMemoryBarrier, VkImageMemoryBarrier
- srcStageMask / dstStageMask : identifier les stages concernés
- srcAccessMask / dstAccessMask : identifier les types d'accès

S26
THÉORIE**Enregistrement Multi-Thread**

- Command pools non thread-safe : un pool par thread
- Secondary command buffers : enregistrés en parallèle, exécutés depuis primary via vkCmdExecuteCommands
- Thread pool : N threads → N secondary CBs → un primary CB les exécute
- Shared resources : les VkImage, VkBuffer sont thread-safe en lecture
- Parallel recording : découper la scène par zone ou par material group
- Timeline semaphores (Vulkan 1.2) : synchronisation fine multi-frame

S27
TP**TP — Parallel Command Recording**

- Thread pool simple (4 workers)
- Chaque thread possède son VkCommandPool et son secondary CB
- Scène de 500 meshes : découper en 4 groupes → 4 threads
- Primary CB : vkCmdExecuteCommands avec les 4 secondary CBs
- Benchmark : single-thread vs multi-thread recording
- Timeline semaphores pour la synchronisation inter-frames

Livrable : Enregistrement parallèle : 4 threads + 500 meshes à 60 FPS

S28
THÉORIE**Compute Pipeline Vulkan**

- VkComputePipeline : plus simple que le graphique (juste 1 shader stage)
- Compute shader GLSL : layout(local_size_x=256) in
- SSBO (Storage Buffer) : VK_DESCRIPTOR_TYPE_STORAGE_BUFFER, lecture-écriture
- Storage Images : VK_DESCRIPTOR_TYPE_STORAGE_IMAGE, imageLoad/imageStore
- vkCmdDispatch(groupsX, groupsY, groupsZ)
- Compute queue séparée : parallèle avec la graphics queue
- Memory barriers compute→graphics : VK_PIPELINE_STAGE_COMPUTE_SHADER_BIT

S29
TP**TP — Simulation de Particules Vulkan**

- Struct Particle { NkVec4f pos; NkVec4f vel; } — 1M particules
- Double-buffering SSBO : current + previous frame
- Compute shader : intégration + collision + reset
- Compute → Graphics barrier : SSBO écrit par compute → lu par vertex shader
- Rendu : vertex shader lit les positions depuis le SSBO (gl_VertexIndex)
- Async compute : compute et graphics en parallèle sur queues séparées

Livrable : 1M particules simulées et rendues entièrement sur GPU Vulkan

S30
THÉORIE**Raytracing Vulkan — Théorie**

- Extensions : VK_KHR_ray_tracing_pipeline, VK_KHR_acceleration_structure
- BLAS (Bottom-Level Acceleration Structure) : géométrie (triangles)
- TLAS (Top-Level Acceleration Structure) : instances de BLAS dans le monde
- Shader stages RT : rgen (ray generation), rchit (closest hit), rmiss, rahit, rint
- Shader Binding Table (SBT) : mappe les shaders aux instances
- vkCmdTraceRaysKHR : lancer les rays depuis le rgen shader
- Ray payload : données échangées entre les shaders RT

S31
TP**TP — Ray Traced Shadows**

- Activer les features RT : VkPhysicalDeviceRayTracingPipelineFeaturesKHR
- Créer BLAS pour chaque mesh
- Créer TLAS avec les instances (transform 3D)
- Shader rgen : lancer des shadow rays depuis le G-Buffer
- Shader rmiss : aucun obstacle → pas d'ombre
- Shader rchit : obstacle → ombre
- Hybride : raytracing pour les ombres, rasterisation pour le reste

Livrable : Ombres raytraced hybrides dans une scène rastérisée Vulkan

Description

Projet individuel ou binôme. Livraison : code source + démo interactive + rapport technique (15 pages minimum).

Exigences Minimales

1. Instance Vulkan 1.2+, validation layers, device selection robuste
2. Swapchain avec gestion de redimensionnement
3. VkGraphicsPipeline + shaders SPIR-V (GLSL compilé via glslc/Jenga)
4. Vertex/index buffers en DEVICE_LOCAL (staging buffer)
5. Descriptors : UBO caméra + push constants par objet
6. Textures : VkImage + VkImageView + VkSampler + mipmaps
7. NKMath complète : NkVec3/4f, NkMat4x4, NkQuaternion, NkTransform3D
8. Éclairage Phong ou PBR en GLSL SPIR-V
9. Depth buffer + shadow map
10. Multi-threading : secondary command buffers
11. 60 FPS stable en 1280×720

Bonus

- Compute shader : particules ou culling GPU
- Raytracing hybride : ombres ou réflexions
- G-Buffer (deferred shading)
- Timeline semaphores + async compute
- PBR complet + IBL

Grille d'Évaluation

Critère	Points	Vérification
Instance + Device + Swapchain	/15	vkCreateDevice, swapchain resize
Pipeline graphique + shaders SPIR-V	/15	glslc via Jenga, PSO complet
Buffers + mémoire GPU (staging correct)	/10	Profiler GPU

Descriptors + UBO + Push Constants	/10	MVP fonctionnel
Textures + mipmaps + sampler	/10	Scène texturée
NKMath complète (6 types)	/10	Chaque type testé
Éclairage Phong ou PBR	/10	3 lumières minimum
Synchronisation + multi-threading	/10	Validation layers : 0 erreur
Rapport technique (15 pages)	/10	Livré en PDF
BONUS : RT + compute + deferred	+15	Benchmark documenté

Évaluation Globale

Critère	Poids	Modalité
TPs hebdomadaires (code + démo + NKMath)	10 %	Continu
Examens théoriques SN	40 %	
Projet final — code + NKMath complète	20 %	
Soutenance + rapport	20 %	
CC	20%	